

Türkiye House Price Prediction

A reproducible regression study and a corrected web application

6 October 2026 · Repository-based evaluation · Six-page report

Executive summary

This project estimates residential property prices from nine features using a Python machine learning pipeline and a browser interface. The correction replaces inconsistent notebook preprocessing with one reproducible workflow. Four regressors are compared on the same group-separated partitions; XGBoost is selected by validation log-price RMSE before the held-out test set is evaluated.

EVIDENCE	MEASURED VALUE
Eligible records / held-out test	17,292 / 3,455
Selected model / log-price R^2	XGBoost / 0.671
Mean absolute error in lira	TRY 742,473
Median absolute percentage error	16.2%

What changed

The API now starts independently of the working directory, validates supported inputs, converts building age from years to the training bands, and uses one categorical vocabulary shared with the form. The three controls that never affected predictions have been removed. The fitted artifact now includes learned preprocessing, so inference applies the same transforms used during training.

System flow

BROWSER	API	FITTED PIPELINE
Nine validated fields Same-origin request	Validate → encode age Build ordered DataFrame	Impute → group → encode Predict log price → expm1

Interpretation and scope

The application is a research demonstration, not a live valuation service. The source and collection date of the included data are undocumented, location coverage is highly uneven, and properties outside the filtering rules are not represented by the reported evaluation. The error figures should accompany any portfolio claim about model quality.

Evidence: Model/metrics.json, training/train.py and the corrected Backend/ and Frontend/ modules. All figures in this report are newly measured unless explicitly identified as historical.

01 Data audit and coverage

The raw CSV contains 20,326 rows and 15 columns, with 195 exact duplicate rows. Deterministic cleaning retains 17,292 rows across 53 city labels. The raw file is preserved; both processed snapshots are now generated from it by the same code.

FIXED ELIGIBILITY RULE	VALUE
Net / gross area	50 < net < 500 m ² ; 50 < gross < 600 m ²
Rooms / bathrooms	0 < rooms < 9; 0 < bathrooms < 5; missing values retained
Building / price	Positive total floors; TRY 1M < price < TRY 20M

These inherited bounds define the modeled segment. They exclude cheaper and luxury properties and can change observed performance. No model-specific price/area filtering is applied.

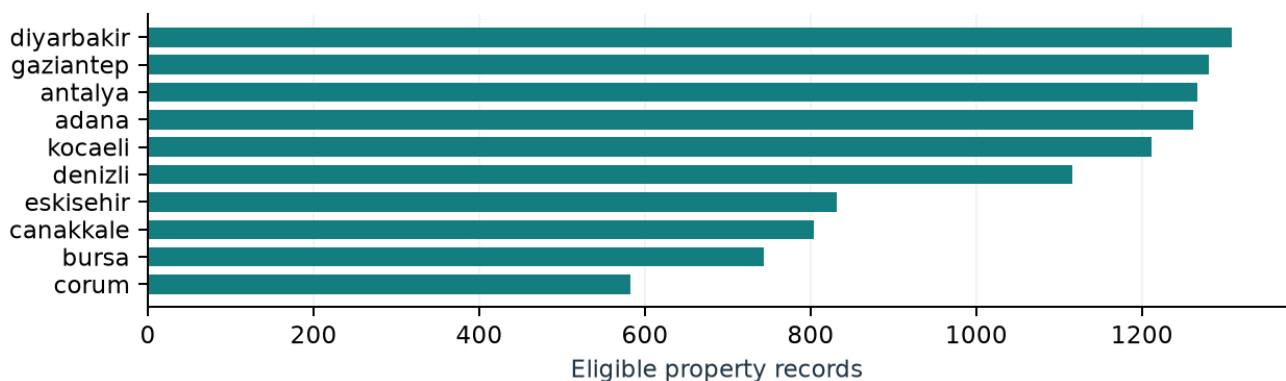


Figure 1. Ten most represented cities after cleaning. Counts describe the repository file, not the national housing stock.

Twelve cities have fewer than 30 eligible records. İstanbul has 4 eligible records (8 raw), Ankara has 1 and Hakkari has 1. A city appearing in a dropdown is therefore not evidence of a reliable city-specific estimate.

Missing data and feature selection

Among retained records, floor is missing in 1,553 rows, rooms in 35, and bathrooms in 43. The fitted pipeline fills numeric inputs with development-set medians and represents a missing categorical value explicitly. It never uses the target price to impute a predictor. The online form requires complete inputs.

Gross area is excluded because it overlaps strongly with net area. Deed status, furnishing, investment suitability and swap availability are not part of the nine-feature serving contract. Cleaning retains the original columns for audit; the EDA snapshot contains nine deterministic predictors plus $\log_{1p}(\text{price})$, with missing values still visible.

Unresolved provenance: the repository does not identify a source URL, collection period, listing-versus-sale status or data license. This report cannot establish those facts from the CSV alone.

02 Experimental design

Holdout isolation

Exact duplicates are removed first. Rows with identical nine-feature profiles are grouped before splitting, so duplicate input profiles cannot appear in multiple partitions. The first GroupShuffleSplit uses seed 42 and reserves 20% of groups for testing. A second split, seed 43, allocates one quarter of the remaining groups to validation. Group proportions produce approximately 60/20/20 row proportions.

PARTITION	ROWS	ROLE
Training	10,375	Fit candidate pipelines
Validation	3,462	Select lowest log-RMSE
Test	3,455	Final evaluation only

The file contains 16,192 distinct feature profiles. Audited overlap is zero for train/validation, train/test and validation/test. After choosing the winner, each candidate is fitted afresh on the combined 13,837 development rows and evaluated on the same untouched test rows. The selected development-fitted pipeline becomes the serving artifact.

Shared preprocessing

Building age is a deterministic six-band feature: new = 0; 1-4 years = 1; 5-10 = 2; 11-15 = 3; 16-20 = 4; and 21+ = 5. Numeric medians and heating categories occurring at least 500 times are learned only inside pipeline.fit. Other heating types become Diğər. The target is log1p(price).

Linear Regression, Random Forest and XGBoost use robust-scaled numeric features and one-hot categorical features, with unknown categories ignored by the encoder. CatBoost uses the same cleaned feature frame with native categorical handling. The fitted preprocessing is serialized with the estimator. [1, 2]

MODEL	FIXED SETTINGS
Linear Regression	Ordinary least squares after shared encoding
Random Forest	200 trees; minimum leaf size 3; seed 42
XGBoost	400 trees; depth 5; learning rate 0.05; row/column sampling 0.8; L2 = 10; seed 42
CatBoost	600 iterations; depth 6; learning rate 0.05; RMSE loss; seed 42

Settings are fixed in training/train.py. No new hyperparameter search is claimed. Model selection uses validation RMSE only; showing final test comparisons does not change the selected estimator.

03 Results and error analysis

XGBoost had the lowest validation log-RMSE (0.3044), followed by CatBoost (0.3079), Random Forest (0.3212) and Linear Regression (0.3293). The final comparison below uses exactly the same held-out records for every model.

MODEL	LOG RMSE	LOG R ²	MAE (TRY)	RMSE (TRY)
Linear Regression	0.3242	0.6147	806,299	1,559,673
Random Forest	0.3117	0.6439	767,432	1,495,169
XGBoost	0.2996	0.6709	742,473	1,442,521
CatBoost	0.3027	0.6641	745,324	1,438,074

Table 1. Held-out results after development-set refitting. RMSE = square root of mean squared error; MAE = mean absolute error. R² measures explained variance relative to a constant predictor; it is not percentage accuracy.

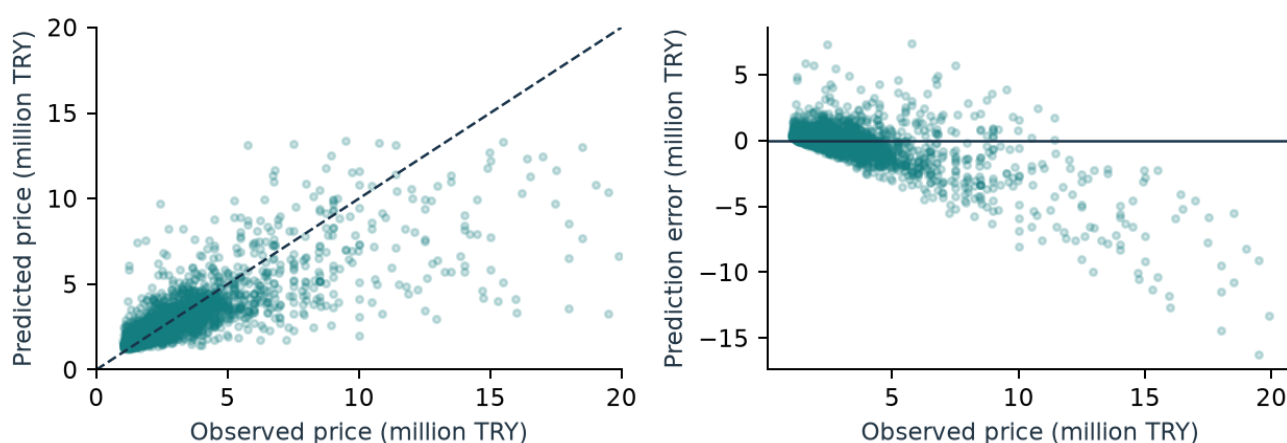


Figure 2. XGBoost observed versus predicted price (left) and signed residuals (right). Negative residuals mean underestimation. Prices and errors are in millions of Turkish lira.

What the measurements imply

The selected pipeline has a median absolute percentage error of 16.2%. This is the median of individual $|\text{prediction} - \text{observation}| / \text{observation}$ ratios; it is not an average error or confidence interval. Its lira-space R² is 0.594, lower than its log-space R² of 0.671. Large-price errors matter more on the original currency scale.

CatBoost has a slightly lower lira RMSE than XGBoost, while XGBoost has lower MAE and log-RMSE. There is no statistically established universal winner from one partition. XGBoost remains selected because the decision rule was validation log-RMSE, not whichever test metric looks best afterward.

The plots show larger underestimates toward the expensive end. Training on log prices and directly exponentiating predictions does not estimate the arithmetic conditional mean without additional assumptions or calibration. No uncertainty intervals or bias correction are claimed.

Historical notebook scores are not pooled with these results: earlier experiments used different filters and splits and included preprocessing leakage risks. This report uses the new shared protocol exclusively.

04 Application corrections and verification

ORIGINAL PROBLEM	IMPLEMENTED CORRECTION
Startup depended on current directory	Package imports, root app.py entrypoint and source-relative artifact/static paths
Age in years treated as an encoded category	One shared conversion maps actual years to six training bands
Category spelling drift	API validates one vocabulary; browser loads options from /options
Unvalidated or empty form values	Required/bounded HTML inputs plus server-side finite-value, range and category checks
Ignored checkboxes and stale results	Removed unused flags; hide old results on new requests; show progress/errors and permit retry
Different local/remote API addresses	Frontend and API share a domain; relative requests replace the Render URL
Preprocessing leakage and divergent snapshots	Training-only fitted pipeline, shared split protocol, regenerated CSVs and executable notebook summaries
Unpinned production dependencies	Pinned runtime graph; Linux CPU-only XGBoost; training-only libraries excluded from serving requirements

Verified behavior

During repair, 16 Python tests passed: age boundaries, feature order, startup from different directories, valid/invalid API inputs, train-only medians, dataset regeneration and non-overlapping feature groups. Initial checks reproduced the original failures before correction. The test harness is not included in this simplified repository.

An isolated Chromium run verified required-field blocking, Enter-key submission, a real successful API response, recovery from a simulated HTTP 503, and a 390-pixel mobile viewport without horizontal overflow. Desktop and mobile screenshots were inspected; no page exceptions occurred. All four notebooks' code cells also executed successfully.

Public API

GET / serves the form; **GET /health** checks successful startup; **GET /options** returns accepted categories; **POST /predict** returns a positive finite lira estimate. Interactive documentation is at **/docs**. Input fields are city, net_area, rooms, bathrooms, heating, floor, total_floors, building_age (years), and usage_status.

Contract change: obsolete furnished/investment_suitable/swap_available fields are rejected instead of silently ignored. Clients must use the new nine-field request. API version is 2.0.0.

05 Reproducibility, deployment and limits

Reproduce this evaluation

Use Python 3.12 and install requirements.txt, then install catboost==1.2.10 for model comparison. Run **python -m training.train**. Start the site with **uvicorn app:app**. macOS needs OpenMP for XGBoost; the verified workspace alternative using scikit-learn's bundled library is documented in DEPLOYMENT.md.

training/data.py rebuilds the processed CSVs from the preserved raw file. training/train.py trains all four candidates, selects by validation error and saves Model/best_model.pkl, Model/input_options.json and Model/metrics.json. The metrics JSON includes scores, split row IDs, seeds, package versions and raw/model SHA-256 hashes. The serving artifact excludes test rows from fitting.

Recorded environment: Python 3.12.9; NumPy 2.5.3; pandas 3.0.6; scikit-learn 1.7.2; CatBoost 1.2.10; XGBoost 3.4.1. The notebooks use the shared training functions and Model/metrics.json.

Vercel preparation

The repository includes app.py, .python-version, root requirements and vercel.json. Static frontend files and the Python API use one origin. The 24 downloaded Linux/Python 3.12 runtime wheels total about 275.9 MB uncompressed before model, bytecode and platform overhead. This is a dependency-size estimate, not a successful Vercel build. [3, 4]

A live deployment is still required to verify availability, memory, cold-start latency and the final bundle. No cloud settings were changed. The earlier Netlify connection reset occurred during TLS, before application code ran; repository repairs cannot prove that separate network/hosting fault is resolved. Hobby is intended for personal, non-commercial use within quotas. [5]

Limitations that code cannot remove

The sample has unknown provenance and age, and lacks verified sale outcomes, district/neighborhood detail, property condition and time variables. City coverage is highly imbalanced. Grouped random holdouts reduce identical-profile overlap but do not demonstrate future-market or unseen-city generalization. Records may still represent related listings that cannot be identified without IDs. Fixed price bounds narrow the evaluated population. Single-split results provide no confidence interval.

Next evidence priorities are to establish the dataset source and permissions, collect dated geographically balanced records, then validate temporal and geographic generalization. Present the application as a reproducible historical-data demonstration and publish its error metrics alongside examples.

References and repository evidence

[1] scikit-learn. Common pitfalls: preprocessing consistency and leakage. https://scikit-learn.org/stable/common_pitfalls.html

[2] CatBoost. CatBoostRegressor reference. https://catboost.ai/docs/en/concepts/python-reference_catboostregressor

[3] Vercel. FastAPI deployment. <https://vercel.com/docs/frameworks/backend/fastapi>

[4] Vercel. Function limits. <https://vercel.com/docs/functions/limitations>

[5] Vercel. Hobby plan. <https://vercel.com/docs/plans/hobby>

Primary evidence: data/raw/home_price.csv; training/data.py; training/train.py; Model/metrics.json. Documentation checked 6 October 2026. Regression and browser checks were run during repair; supporting test artifacts are omitted from the simplified repository.